Erlind Sejdiu
106357

SYCL

Portable heterogene Programmierung mit C++

Problemstellung, Anwendungsfälle und Programmiermodell

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: [Name der betreuenden Person]
Themengruppe: [Themengruppe des Seminars]
Abgabedatum: [Datum]

Inhaltsverzeichnis

1 Problemstellung **1**

2 Forschungsfrage **1**

3 Stand der Technik **2**

4 Methode **2**

5 Untersuchung **3**

5.1 Stufe 1: Der erste Kernel 4

5.2 Stufe 2: Expliziter Gerätespeicher 4

5.3 Stufe 3: Kachelung im lokalen Speicher 5

5.4 Stufe 4: Gerätewechsel und Kachelgröße 7

6 Ergebnisse **7**

7 Diskussion **8**

8 Schlussfolgerung **8**

Literaturverzeichnis **10**

Abbildungsverzeichnis

1 Getrennte Codebasen je Hersteller (a) und eine SYCL-Codebasis für alle Backends (b) 1
2 Abhängigkeitsgraph: Die Kopie von D kann parallel zu Kernel 1 laufen 5
3 Speicherhierarchie von Host und Gerät und die Wege des Datentransfers 5
4 Gliederung des Indexraums in Work-Groups und Work-Items 6
5 Typische Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU 6
6 Synchrone Fehler werden beim Einreichen geworfen, asynchrone erst, wenn der Host wartet 7
7 Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b) 8

Tabellenverzeichnis

1 SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt) 2
2 Stufen der Portierung und die dabei eingeführten Konzepte 3
3 Die beiden Speichermodelle von SYCL 5

1 Problemstellung

Rechenleistung liegt heutzutage in verschiedenen Formen vor. Neben der CPU stehen in fast jedem Rechner GPUs und andere technische Komponenten zur Verfügung, die datenparallele Aufgaben schneller bearbeiten können. Die Software, um diese Geräte jeweils anzusprechen, kann variieren, denn jeder Hersteller bringt sein eigenes Programmiermodell mit. Für die Software ergeben sich daraus drei Probleme:

1. **Herstellerbindung:** CUDA läuft nur auf NVIDIA-GPUs [1]; AMD (HIP) und Intel (Level Zero) haben eigene Schnittstellen.
2. **Mehrfache Codebasen:** Wer mehrere Hersteller unterstützen will, pflegt mehrere Implementierungen parallel (Abbildung 1a).
3. **Abstraktion gegen Kontrolle:** Offene Alternativen sind entweder sehr hardwarenah und aufwendig wie OpenCL [2] oder lassen wie OpenMP-Direktiven wenig Kontrolle über Speicher und Ausführung.

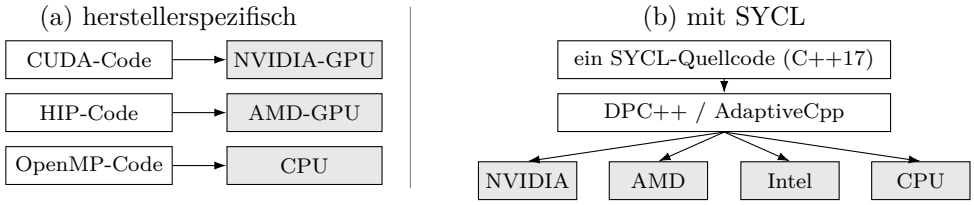


Abbildung 1: Getrennte Codebasen je Hersteller (a) und eine SYCL-Codebasis für alle Backends (b)

2 Forschungsfrage

SYCL ist ein offener Standard der Khronos Group, der diese oben genannten Probleme lösen will. Mithilfe eines heterogenen Codes in Standard-C++ soll auf CPUs und GPUs verschiedener Hersteller dasselbe Programm laufen [3]. Ob ein Programm überall läuft, ist jedoch etwas anderes, als ob es überall schnell läuft. Diese Arbeit untersucht deshalb:

> Wie adressiert das Programmiermodell von SYCL die drei genannten Probleme, und erreicht ein schrittweise nach SYCL portiertes Programm neben funktionaler Portabilität auch Performance-Portabilität zwischen CPU und GPU?

Daraus ergeben sich zwei Teilfragen. (F1) Welche Konzepte des Programmiermodells werden bei einer Portierung benötigt, und welches der drei Probleme lösen sie? (F2) Welche Stufen einer Portierung lassen sich unverändert auf verschiedenen Geräten ausführen, und welche Parameter bleiben gerätespezifisch?

3 Stand der Technik

SYCL basiert auf C++17 [3]. Konkrete Implementierungen des Standards wie DPC++ [4] und AdaptiveCpp [5], [6] bilden die Grundlage über Backends wie CUDA, HIP, Level Zero, OpenCL oder OpenMP auf die Hardware ab und übersetzen den Gerätecode meist erst beim ersten Start just in time für das konkrete Gerät [7]. Den drei Problemen setzt SYCL vier Entwurfsziele entgegen. Portabilität (ein Quellcode, viele Backends), Offenheit (ein Standard mit mehreren unabhängigen Implementierungen), Produktivität (Host- und Gerätecode in einer Datei in Standard-C++) und Kontrolle (Hardwarekonzepte wie lokaler Speicher bleiben zugänglich). Tabelle 1 ordnet SYCL entlang dieser Ziele ein. Kokkos [8] und RAJA [9] verfolgen ähnliche Ziele, sind aber kein offener Industriestandard.

	herstellerunabh.	**offener Standard**	**Standard-C++**	**Kontrolle**
CUDA	–	–	(✓)	✓
OpenCL	✓	✓	–	✓
OpenMP-Offload	✓	✓	✓	(✓)
Kokkos/RAJA	✓	–	✓	(✓)
SYCL	✓	✓	✓	✓

Tabelle 1: SYCL im Vergleich zu anderen Programmiermodellen ((✓) = eingeschränkt)

Eingesetzt wird SYCL vor allem im wissenschaftlichen Rechnen auf Systemen verschiedener Hersteller, etwa auf dem Supercomputer Aurora oder in GROMACS, bei der Migration von CUDA-Anwendungen mit dem Werkzeug SYCLomatic und in portablen Bibliotheken wie oneMKL [10]. Frühere Studien fanden SYCL auf vielen Plattformen konkurrenzfähig mit herstellerspezifischem Code, aber deutlich abhängig von Implementierung und Parametrierung [11], [12]. Als Maß für Performance-Portabilität hat sich die Kennzahl von Pennycook, Sewall und Lee etabliert [13]. Offen bleibt in diesen Arbeiten meist, welcher Schritt einer Portierung welchen Effekt auf welchem Gerät hat. Hier setzt folgende Seminararbeit an.

4 Methode

Untersucht wird die Multiplikation $C = A \cdot B$ zweier $n \times n$-Matrizen in einfacher Genauigkeit [14]. Das Problem benötigt $2n^3$ Gleitkommaoperationen bei $3n^2$ Werten Datenvolumen, ist gut parallelisierbar und leicht auf Korrektheit prüfbar. Ein sequentielles C++-Programm wird in Stufen nach SYCL überführt, wobei jede Stufe nur so viele neue Konzepte einführt, wie für ihren Schritt nötig sind (Tabelle 2). Die Konzepte des Programmiermodells werden deshalb in Abschnitt 5 jeweils an der Stufe erklärt, in

der sie zum ersten Mal gebraucht werden. Das beantwortet F1. Zugleich lässt sich die Wirkung jedes Schritts getrennt messen, was F2 beantwortet.

Stufe	Änderung	Neue Konzepte
0	Ausgangsprogramm, drei Schleifen, ein CPU-Kern	–
1	Äußere Schleifen durch `parallel_for` ersetzt	Host und Gerät, Queue, Kernel, `range`, USM shared
2	Getrennter Gerätespeicher, explizites Kopieren	USM device, `memcpy`, Events, Abhängigkeiten, Buffer
3	Kachelung im lokalen Speicher	Command Group, `nd_range`, Work-Group, lokaler Speicher, Barriere
4	Kein neuer Code: Gerätewechsel und Variation der Kachelgröße	Geräteauswahl, Backends, Geräteabfragen, Fehlerbehandlung

Tabelle 2: Stufen der Portierung und die dabei eingeführten Konzepte

Bewertet werden zwei Ebenen: funktionale Portabilität (das Programm läuft unverändert und korrekt auf jedem Gerät) und Performance-Portabilität. Letztere wird als harmonischer Mittelwert der Effizienzen e_i über die Gerätemenge H berechnet [13]:

$$\mathrm{PP}(a,p,H) = \begin{cases} |H| \,\Big/ \sum_{i\in H} 1/e_i(a,p) & \text{falls } a \text{ auf allen } i \in H \text{ läuft,} \\ 0 & \text{sonst.} \end{cases}$$

Als Effizienz dient die Anwendungseffizienz, also die Leistung relativ zur besten gemessenen Variante auf dem jeweiligen Gerät.

Gemessen wird auf einem AMD Ryzen 5 5600X (sechs Kerne, OpenCL-Backend) und einer NVIDIA GeForce RTX 5060 Ti (CUDA-Backend) mit DPC++ [4]. Jede Stufe läuft für $n \in \{256, 512, 1024, 2048\}$; nach einem getrennt erfassten ersten Lauf, der die JIT-Kompilierung enthält, folgen fünf Messläufe, von denen der Median berichtet wird. Kernel- und Transferzeit werden getrennt erfasst, und jedes Ergebnis wird stichprobenartig gegen eine Referenzrechnung in doppelter Genauigkeit geprüft.

5 Untersuchung

Die größte Hürde war die Werkzeugkette: Die Standardinstallation von oneAPI erkannte nur die CPU, für die NVIDIA-GPU war ein eigener Build von DPC++ nötig. Alle Programme wurden einmal für beide Geräte zugleich übersetzt.

5.1 Stufe 1: Der erste Kernel

SYCL unterscheidet den Host, auf dem das Hauptprogramm läuft, und Geräte, auf denen Kernel ausgeführt werden. Die Verbindung zu einem Gerät stellt eine Queue her; ohne weitere Angabe wählt sie das als am leistungsfähigsten eingeschätzte Gerät. Für Stufe 1 genügen daneben zwei weitere Konzepte. Mit `malloc_shared` angelegter Unified Shared Memory (USM) ist auf Host und Gerät über denselben Zeiger gültig; die Laufzeit migriert die Daten bei Bedarf. `parallel_for` führt einen Kernel, hier ein gewöhnliches C++-Lambda, für jeden Punkt eines ein- bis dreidimensionalen Indexraums (`range`) einmal aus; eine einzelne Ausführung heißt Work-Item. Die beiden äußeren Schleifen des Ausgangsprogramms entfallen, die innere Rechenvorschrift wird unverändert übernommen. Host- und Gerätecode stehen in derselben Datei (Single-Source).

Das Einreichen kehrt sofort zurück; die Ausführung ist asynchron, und der Host wartet nur, wenn er es wie hier mit `wait()` verlangt. Für den Gerätecode gelten Einschränkungen, weil er auf Hardware ohne vollständige C++-Laufzeitumgebung laufen muss: Ausnahmen, virtuelle Funktionsaufrufe, Rekursion und dynamische Speicherreservierung sind im Kernel nicht erlaubt, und Daten werden per Wertkopie übernommen [3]. Bei einer einfachen `range` überlässt der Programmierer die Aufteilung der Work-Items vollständig der Implementierung.

5.2 Stufe 2: Expliziter Gerätespeicher

GPUs besitzen meist eigenen Speicher, der vom Hauptspeicher des Hosts getrennt ist. Bei USM shared bleibt verborgen, wann Daten wandern. Stufe 2 legt die Matrizen deshalb mit `malloc_device` nur auf dem Gerät an und kopiert sie mit `q.memcpy` explizit hin und zurück. Neben device und shared gibt es als dritte USM-Art host (im Hauptspeicher, vom Gerät aus lesbar) [10]. So lassen sich Transfer- und Rechenzeit erstmals getrennt messen.

Mit expliziten Kopien wird die Reihenfolge wichtig: Der Kernel darf erst starten, wenn A und B angekommen sind. Jedes Einreichen liefert ein Event, über das sich Abhängigkeiten zu früheren Aufträgen angeben lassen. Aus diesen baut die Laufzeit einen gerichteten azyklischen Abhängigkeitsgraphen auf (Abbildung 2). Voneinander unabhängige Aufträge darf sie parallel ausführen, abhängige startet sie erst, wenn ihre Vorgänger fertig sind [10]. So lassen sich Transfers und Berechnungen überlappen, ohne Threads von Hand zu verwalten. Die Fallstudie wartet der Einfachheit halber nach jedem Schritt.

Bei USM liegt die Verantwortung für diese Abhängigkeiten beim Programmierer. SYCL bietet mit Buffern und Accessoren eine Alternative (Tabelle 3): Ein Buffer kapselt die Daten, und Kernel greifen über Accessoren darauf zu, die angeben, ob gelesen

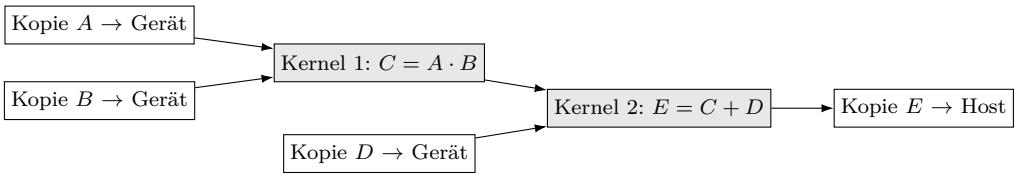


Abbildung 2: Abhängigkeitsgraph: Die Kopie von D kann parallel zu Kernel 1 laufen

oder geschrieben wird. Daraus leitet die Laufzeit Kopien und Abhängigkeiten selbst ab. Die Fallstudie bleibt bei USM, weil es dem Ausgangsprogramm mit seinen Zeigern am nächsten ist.

	Buffer und Accessoren	**Unified Shared Memory (USM)**
Zugriff	über Accessor-Objekte mit Zugriffsmodus	über gewöhnliche Zeiger
Datentransfer	automatisch durch die Laufzeit	shared: automatisch; device: explizit mit `memcpy`
Abhängigkeiten	automatisch aus den Accessoren	selbst über Events oder in-order-Queue
Stärke	sicher, wenig Fehlerquellen	vertraut aus C++, leicht zu portieren

Tabelle 3: Die beiden Speichermodelle von SYCL

5.3 Stufe 3: Kachelung im lokalen Speicher

In Stufe 2 liest jedes Work-Item eine ganze Zeile von A und Spalte von B aus dem langsamen globalen Speicher. Stufe 3 nutzt die Speicherhierarchie des Geräts (Abbildung 3): Neben dem globalen Speicher (für alle Work-Items sichtbar, groß, aber langsam) gibt es lokalen Speicher (pro Work-Group, klein und schnell) und privaten Speicher (Register). Diese Hierarchie gezielt zu nutzen ist, wie bei CUDA, der wichtigste Hebel für Leistung [14].

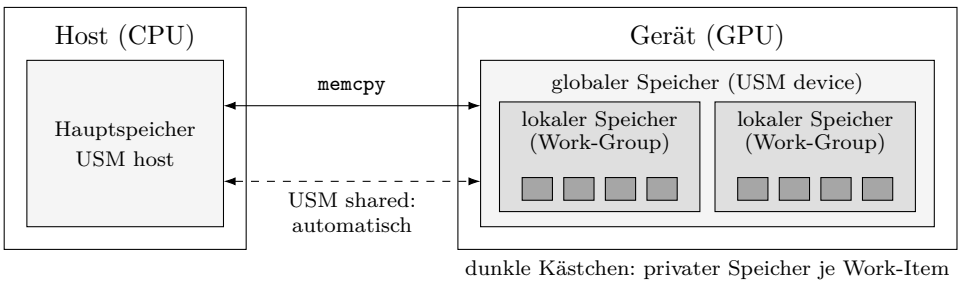


Abbildung 3: Speicherhierarchie von Host und Gerät und die Wege des Datentransfers

Lokaler Speicher setzt voraus, dass der Programmierer festlegt, welche Work-Items zusammenarbeiten. Statt einer `range` verwendet Stufe 3 deshalb eine `nd_range`, die den Indexraum zusätzlich in Work-Groups der Größe $T \times T$ gliedert (Abbildung 4). Die Work-Items einer Gruppe laufen gemeinsam auf einer Recheneinheit, teilen sich den lokalen Speicher und können sich über Barrieren synchronisieren. Jede Work-Group lädt nacheinander $T \times T$-Kacheln von A und B in den lokalen Speicher, wartet an einer Barriere, bis alle Werte geladen sind, und rechnet dann nur noch mit den schnellen Kopien.

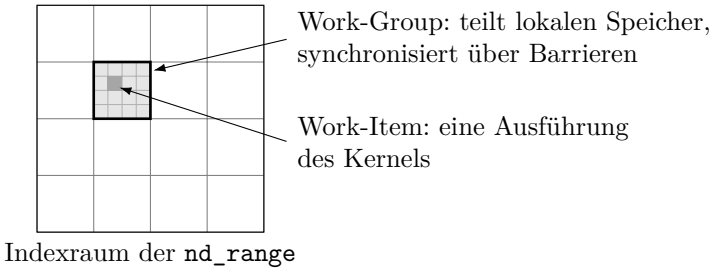


Abbildung 4: Gliederung des Indexraums in Work-Groups und Work-Items

Um lokalen Speicher anzufordern, reicht der vereinfachte Aufruf aus Stufe 1 nicht mehr aus. Die Arbeit wird nun als Command Group mit `q.submit` eingereicht: einer Funktion, die ein Handler-Objekt erhält, über das sie Ressourcen wie den lokalen Speicher (`local_accessor`) und Abhängigkeiten deklariert und genau eine Aktion startet. Der Kernel erhält statt einer `id` ein `nd_item`, das globale und lokale Position sowie die Gruppe liefert.

Diese Hierarchie bildet die Hardware ab (Abbildung 5) und entspricht weitgehend dem Modell von CUDA [1]: Auf einer NVIDIA-GPU wird eine Work-Group zu einem Thread-Block mit höchstens 1024 Threads [15]. Auf einer CPU arbeitet typischerweise ein Kern eine Work-Group ab. Seit SYCL 2020 gibt es darunter zudem Sub-Groups, die einem Warp bzw. den Lanes einer Vektoreinheit entsprechen, sowie fertige Gruppenalgorithmen und Reduktionen [3]. Diese Verwandtschaft erleichtert die Migration aus CUDA.

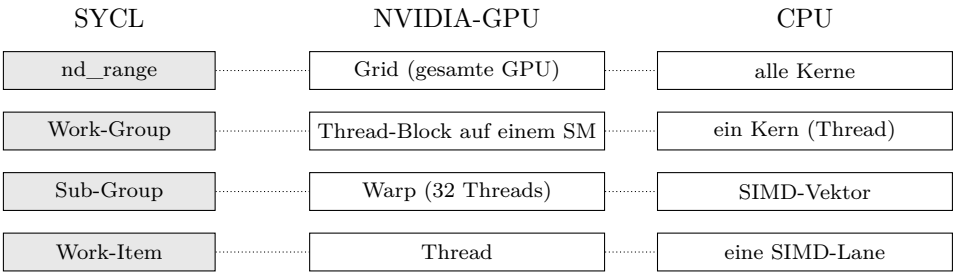


Abbildung 5: Typische Abbildung der SYCL-Ausführungshierarchie auf GPU und CPU

5.4 Stufe 4: Gerätewechsel und Kachelgröße

Stufe 4 enthält keinen neuen Code. Sie prüft, ob die Programme der Stufen 1 bis 3 unverändert auf einem anderen Gerät laufen. Dafür ist die Geräteauswahl nötig: Geräte sind nach Plattformen gruppiert, wobei jede Plattform einem Backend entspricht; auch die CPU tritt als Gerät auf. Welches Gerät eine Queue verwendet, bestimmt ein Device Selector, der jedes Gerät anhand seiner Eigenschaften bewertet und das bestbewertete wählt. Implementierungen erlauben außerdem eine Auswahl von außen; in der Fallstudie wurde nur die Umgebungsvariable `ONEAPI_DEVICE_SELECTOR` zwischen `opencl:cpu` und `cuda:gpu` gewechselt. Dass die CPU ein vollwertiges Gerät ist, macht sie zum Ausweichgerät und erlaubt die Entwicklung ohne GPU.

Zusätzlich wurde Stufe 3 mit den Kachelgrößen $T \in \{8, 16, 32, 64\}$ übersetzt und auf beiden Geräten gemessen. Bei $T = 64$ umfasst eine Work-Group $64 \cdot 64 = 4096$ Work-Items und überschreitet damit die Grenze der NVIDIA-GPU; der Kernel wird beim Einreichen abgewiesen. Solche Fehler meldet SYCL als C++-Ausnahmen (Abbildung 6): synchrone Fehler wie eine ungültige Work-Group-Größe sofort beim Einreichen, asynchrone Fehler, die erst während der Ausführung auf dem Gerät auftreten, gesammelt über einen Async Handler, sobald der Host wartet.

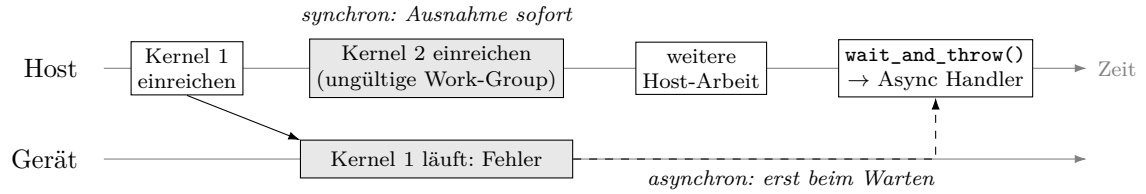


Abbildung 6: Synchrone Fehler werden beim Einreichen geworfen, asynchrone erst, wenn der Host wartet

Um solche Grenzen zu vermeiden, kann ein Programm die maximale Work-Group-Größe zur Laufzeit abfragen und die Kachelgröße über Spezialisierungskonstanten erst dann festlegen [3].

6 Ergebnisse

Alle Stufen liefen ohne Codeänderung und mit korrektem Ergebnis auf beiden Geräten. Abbildung 7 fasst die Messungen zusammen. Bei kleinen Matrizen kostet der Datentransfer zur GPU mehr als die Rechnung; erst ab $n = 1024$ überwiegt die Rechenzeit (a). Die beste Kachelgröße ist geräteabhängig: Auf der GPU ist $T = 16$ am schnellsten, auf der CPU $T = 32$. Bei $T = 64$ startet der Kernel auf der GPU nicht, sodass die Performance-Portabilität auf null fällt (b). Der höchste Wert, PP = 0,99, ergibt sich für $T = 32$.

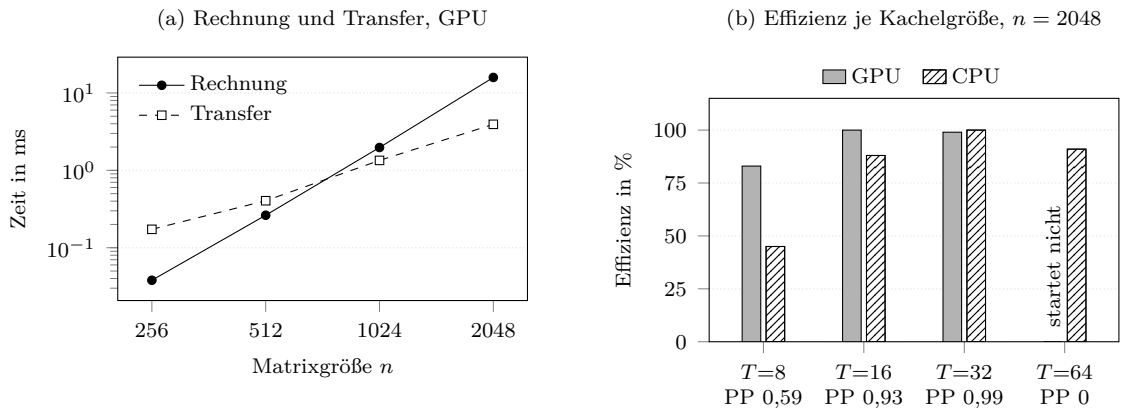


Abbildung 7: Datentransfer (a) und Leistung je Kachelgröße mit Performance-Portabilität (b)

7 Diskussion

Zu F1: Die Stufen zeigen, welches Konzept welches Problem löst. Die Herstellerbindung lösen Backends und Geräteauswahl (Stufe 4), die mehrfachen Codebasen Single-Source in Standard-C++ mit Kerneln als Lambdas (Stufe 1). Den Zielkonflikt zwischen Abstraktion und Kontrolle entschärft SYCL durch einen gestuften Einstieg: Eine einfache `range` mit USM shared genügt für ein lauffähiges Programm, während `nd_range`, lokaler Speicher und expliziter Transfer die Kontrolle für die Optimierung zurückgeben (Stufen 2 und 3).

Zu F2: Die Fallstudie bestätigt funktionale Portabilität für alle Stufen. Performance-Portabilität ergibt sich dagegen nicht von selbst. SYCL abstrahiert die Schnittstelle, nicht die Hardware: Parameter wie die Work-Group-Größe bleiben gerätespezifisch, und ein für die CPU günstiger Wert kann auf der GPU sogar zum Abbruch führen. Dies deckt sich mit der in früheren Studien beobachteten Abhängigkeit von Implementierung und Parametrierung [11].

Die Aussagekraft der Fallstudie ist begrenzt: Untersucht wurden ein einzelner Kernel, zwei Geräte und eine Implementierung. Die PP-Werte gelten nur für diese Gerätemenge [16], und ein Vergleich mit nativem CUDA-Code oder optimierten Bibliotheken fehlt.

8 Schlussfolgerung

SYCL erfüllt sein Versprechen „einmal schreiben, überall ausführen“ für die Korrektheit: Ein schrittweise portiertes Programm lief in allen Stufen unverändert auf CPU und GPU. Die gerätespezifische Abstimmung für hohe Leistung nimmt SYCL dem Programmierer jedoch nicht ab. Folgearbeiten sollten weitere Geräte und eine zweite

Implementierung wie AdaptiveCpp einbeziehen.

Literaturverzeichnis

[1] J. Nickolls, I. Buck, M. Garland und K. Skadron, „Scalable Parallel Programming with CUDA," *ACM Queue*, Jg. 6, Nr. 2, S. 40–53, 2008. DOI: 10.1145/1365490.1365500

[2] J. E. Stone, D. Gohara und G. Shi, „OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," *Computing in Science & Engineering*, Jg. 12, Nr. 3, S. 66–73, 2010. DOI: 10.1109/MCSE.2010.69

[3] The Khronos SYCL Working Group. „SYCL 2020 Specification," besucht am 30. Okt. 2026. Adresse: https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html

[4] Intel Corporation. „Intel LLVM-based Compiler (DPC++) – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/intel/llvm

[5] A. Alpay und V. Heuveline, „SYCL beyond OpenCL: The Architecture, Current State and Future Direction of hipSYCL," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388658

[6] AdaptiveCpp Contributors. „AdaptiveCpp – Repository," besucht am 30. Okt. 2026. Adresse: https://github.com/AdaptiveCpp/AdaptiveCpp

[7] A. Alpay und V. Heuveline, „One Pass to Bind Them: The First Single-Pass SYCL Compiler with Unified Code Representation Across Backends," in *Proceedings of the International Workshop on OpenCL (IWOCL '23)*, 2023.

[8] H. C. Edwards, C. R. Trott und D. Sunderland, „Kokkos: Enabling Manycore Performance Portability through Polymorphic Memory Access Patterns," *Journal of Parallel and Distributed Computing*, Jg. 74, Nr. 12, S. 3202–3216, 2014. DOI: 10.1016/j.jpdc.2014.07.003

[9] D. A. Beckingsale u. a., „RAJA: Portable Performance for Large-Scale Scientific Applications," in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, S. 71–81.

[10] J. Reinders, B. Ashbaugh, J. Brodman, M. Kinsner, J. Pennycook und X. Tian, *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL*, 2. Aufl. Berkeley, CA: Apress, 2023, ISBN: 978-1-4842-9691-2. DOI: 10.1007/978-1-4842-9691-2

[11] T. Deakin und S. McIntosh-Smith, „Evaluating the Performance of HPC-Style SYCL Applications," in *Proceedings of the International Workshop on OpenCL (IWOCL '20)*, 2020. DOI: 10.1145/3388333.3388643

[12] M. Breyer, A. Van Craen und D. Pflüger, „A Comparison of SYCL, OpenCL, CUDA, and OpenMP for Massively Parallel Support Vector Machine Classification on Multi-Vendor Hardware," in *Proceedings of the International Workshop on OpenCL (IWOCL '22)*, 2022.

[13] S. J. Pennycook, J. D. Sewall und V. W. Lee, „A Metric for Performance Portability," in *Proceedings of the 7th International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2016. arXiv: 1611.07409.

[14] W.-m. W. Hwu, D. B. Kirk und I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4. Aufl. Cambridge, MA: Morgan Kaufmann, 2022.

[15] NVIDIA Corporation. „CUDA C++ Programming Guide," besucht am 30. Okt. 2026. Adresse: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`

[16] S. J. Pennycook, J. D. Sewall und V. W. Lee, „Implications of a Metric for Performance Portability," *Future Generation Computer Systems*, Jg. 92, S. 947–958, 2019.